

M8_Unsupervised_learning_Clustering

January 26, 2026

##Unsupervised Learning

Fins ara hem treballat amb models de supervised learning on teníem dades amb etiquetes (spam/no-spam, maligne/benigne, dígit del 0 al 9, etc). En aquests casos l'objectiu era “aprendre” a predir aquestes etiquetes per a noves dades que el model no havia vist mai.

En *unsupervised learning* el model treballa amb dades sense etiquetes, és a dir, només veu les característiques (*features*) però no sap quina és la classe o resposta correcta per a cada exemple. L'objectiu, en aquest cas, no és predir una etiqueta coneguda, sinó descobrir estructures internes a les dades: grups naturals, patrons, anomalies o noves relacions que no coneixíem prèviament.

Entre els principals casos d'ús de l'aprenentatge no supervisat (*unsupervised learning*) trobem:

- Segmentació de clients (a priori no sabem quants tipus de client tenim).
- Detecció d'anomalies (detecció de casos estranys o *outliers*)
- Recomenadors (agrupació de productes similars)

###Clustering

El *clustering* és una tècnica que agrupa dades similars en conjunts (*clusters*). L'objectiu és que els elements dins d'un *cluster* siguin molt similars entre ells, i molt diferents dels elements d'altres *clusters*.

Un dels usos més extesos del *clustering* és la segmentació d'usuaris.

A continuació mostrem un exemple (amb dades sintètiques) per il·lustrar el funcionament d'un dels models més simples i típics per aquest fi: K-means.

###Exemple: clients de supermercat (dades sintètiques)

Una cadena de supermercats té una base de dades de 200 clients amb els seus ingressos anuals i la mitja de la seva despesa setmanal.

Podem agrupar els clients per fer campanyes de marketing personalitzades?

Amb el *clustering* intentarem trobar patrons en les dades per descobrir quants tipus de clients tenim i quines són les seves característiques.

```
[ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
```

```
from sklearn.metrics import silhouette_score
```

Creem dades de 4 tipus de clients de forma sintètica per fer la demostració de la tècnica del clustering.

```
[ ]: # Creem 4 tipus de clients
clients_basics = np.random.randn(60, 2) * [8, 12] + [25, 25]
clients_premiums = np.random.randn(50, 2) * [12, 10] + [75, 80]
clients_estalviadors = np.random.randn(45, 2) * [10, 8] + [50, 30]
clients_esporadics = np.random.randn(45, 2) * [7, 15] + [30, 75]

X_clients = np.vstack([clients_basics, clients_premiums,
                        clients_estalviadors, clients_esporadics])

df_clients = pd.DataFrame(X_clients, columns=['Ingressos (k€)', 'Despesa (€)'])
```

Observem una mostra de les dades generades:

```
[ ]: df_clients.head()
```

```
[ ]: Ingressos (k€)  Despesa (€)
0      22.944136    10.106734
1      29.086879    12.237198
2      27.811701    30.224898
3      22.103786    37.031570
4      17.062463    27.853903
```

Obtenim un seguit d'estadístiques bàsiques:

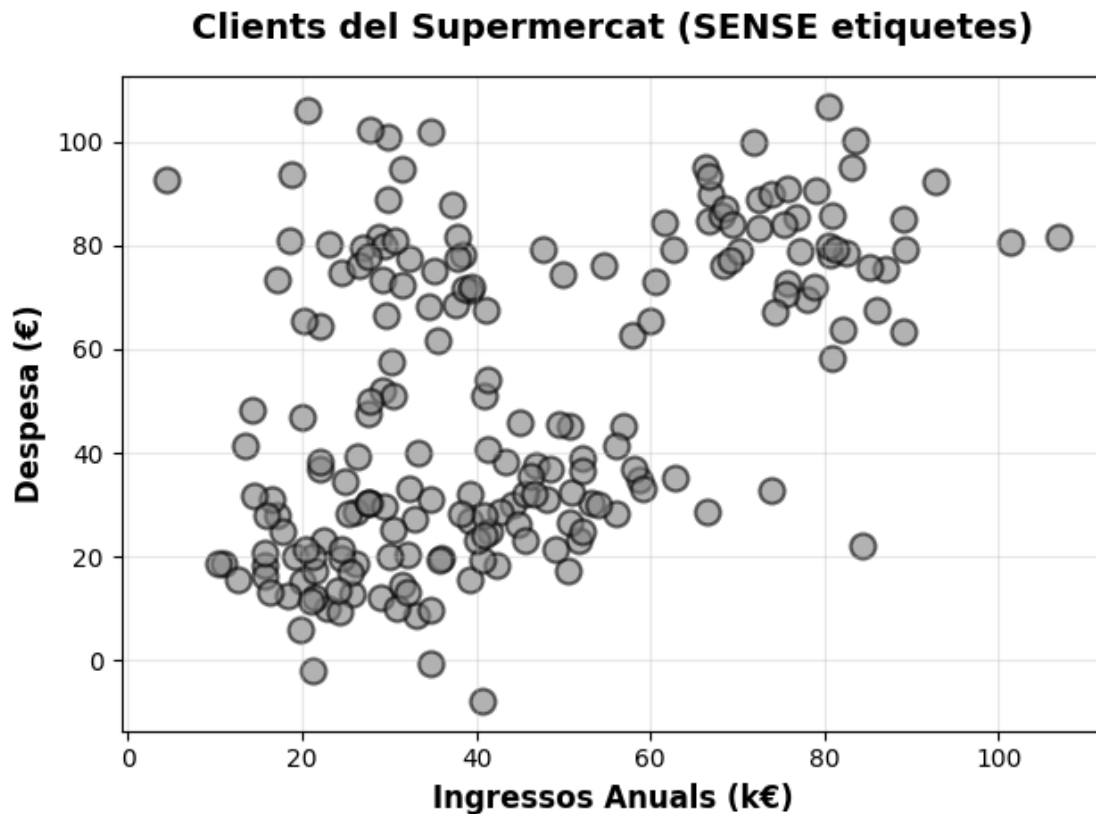
```
[ ]: df_clients.describe().round(2)
```

```
[ ]: Ingressos (k€)  Despesa (€)
count      200.00    200.00
mean       44.23    50.22
std        22.37    28.96
min         4.37   -7.81
25%        26.92    24.99
50%        39.19    43.27
75%        60.17    77.98
max       106.88   106.91
```

Anem a representar les dades:

```
[ ]: plt.scatter(df_clients['Ingressos (k€)'], df_clients['Despesa (€)'],
                 c='gray', s=100, alpha=0.6, edgecolors='black', linewidth=1.5)
plt.xlabel('Ingressos Anuals (k€)', fontsize=12, fontweight='bold')
plt.ylabel('Despesa (€)', fontsize=12, fontweight='bold')
plt.title('Clients del Supermercat (SENSE etiquetes)',
          fontsize=14, fontweight='bold', pad=15)
```

```
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



Quants grups hi veus? On són els límits entre aquests grups? Quin comportament els caracteritza? Totes aquestes preguntes les intentarem respondre.

###K-MEANS

Inicialment indiquem al model quants grups volem (indicat pel paràmetre K). El model escull K punts aleatoris per a centres inicials dels clusters i assigna cada valor al centre més proper. Amb totes els valors assignats recalcula els nous centres de cada cluster i repeteix el procés fins a que els centres es mantenen estables (no varien).

La principal avantatge és que és un model molt ràpid i simple però li hem d'especificar el nombre de grups que volem (K).

Podem trobar el K valor òptim? Doncs a través del **Elbow Method** i provant amb diferents valors de K podem identificar gràficament aquest valor.

Per cada K, calcula la “inèrcia” (suma de distàncies al centre). A major K aquest valor sempre baixa però busquem el “colze” en la gràfica que és el punt on la caiguda es frena: equilibri entre compactació i nombre de grups òptim (punt on que encara que augmentem k, la millora del valor d’inèrcia no és significativa).

```
[ ]: #Normalitzem les dades
scaler = StandardScaler()
X_clients_scaled = scaler.fit_transform(X_clients)

#Càlcul per a diferents valors de K
K_range = range(2, 9)
inertias = []
silhouettes = []

for k in K_range:
    kmeans_temp = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans_temp.fit(X_clients_scaled)
    inertias.append(kmeans_temp.inertia_)
    silhouettes.append(silhouette_score(X_clients_scaled, kmeans_temp.labels_))
```

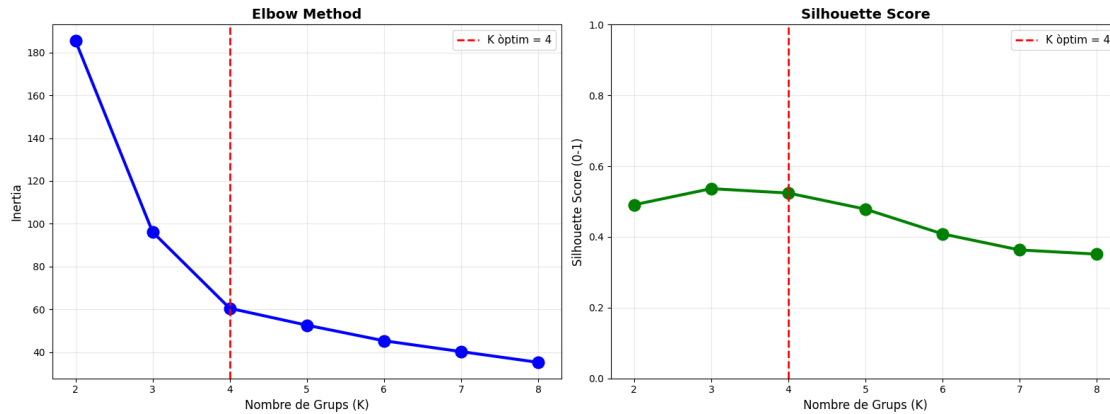
La **silhouette score** és un valor de 0 a 1 que avalua la qualitat dels clústers en K-means mesurant per cada punt de dades: cohesió (punts a prop dins del seu clúster) + separació (distància als clústers veïns):

* >0.5 indiquen bons clústers * 0.2-0.5 acceptables * <0.2 suggereixen revisar K o les dades.

```
[ ]: fig, axes = plt.subplots(1, 2, figsize=(16, 6))

axes[0].plot(K_range, inertias, 'o-', linewidth=3, markersize=12, color='blue')
axes[0].axvline(x=4, color='red', linestyle='--', linewidth=2, label='K òptim = 4')
axes[0].set_xlabel('Nombre de Grups (K)', fontsize=12)
axes[0].set_ylabel('Inertia', fontsize=12)
axes[0].set_title('Elbow Method', fontsize=14, fontweight='bold')
axes[0].legend(fontsize=11)
axes[0].grid(alpha=0.3)
axes[1].plot(K_range, silhouettes, 'o-', linewidth=3, markersize=12, color='green')
axes[1].axvline(x=4, color='red', linestyle='--', linewidth=2, label='K òptim = 4')
axes[1].set_xlabel('Nombre de Grups (K)', fontsize=12)
axes[1].set_ylabel('Silhouette Score (0-1)', fontsize=12)
axes[1].set_title('Silhouette Score', fontsize=14, fontweight='bold')
axes[1].legend(fontsize=11)
axes[1].grid(alpha=0.3)
axes[1].set_ylim([0, 1])

plt.tight_layout()
plt.show()
```



```
[ ]: # K = 4

kmeans_final = KMeans(n_clusters=4, random_state=42, n_init=10)
df_clients['Cluster'] = kmeans_final.fit_predict(X_clients_scaled)

centers_original = scaler.inverse_transform(kmeans_final.cluster_centers_)

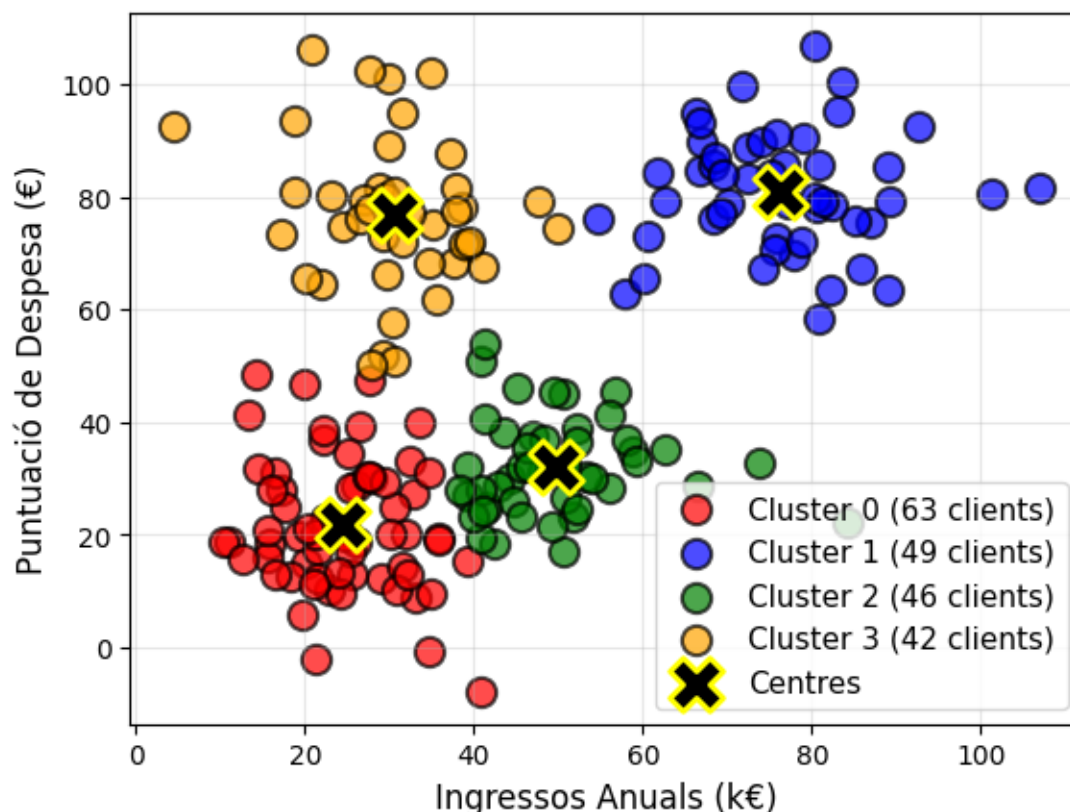
[ ]: colors = ['red', 'blue', 'green', 'orange']

for i in range(4):
    cluster_data = df_clients[df_clients['Cluster'] == i]
    plt.scatter(cluster_data['Ingressos (k€)'], cluster_data['Despesa (€)'],
                s=120, alpha=0.7, c=colors[i], edgecolors='black', linewidth=1.5,
                label=f'Cluster {i} ({len(cluster_data)} clients)')

plt.scatter(centers_original[:, 0], centers_original[:, 1],
            c='black', s=400, marker='X', edgecolors='yellow', linewidth=2,
            label='Centres', zorder=5)

plt.xlabel('Ingressos Anuals (k€)', fontsize=12)
plt.ylabel('Puntuació de Despesa (€)', fontsize=12)
plt.title('CLUSTERING: 4 Grups Descoberts', fontsize=16, fontweight='bold',
         pad=20)
plt.legend(fontsize=11, loc='lower right')
plt.grid(alpha=0.3)
plt.show()
```

CLUSTERING: 4 Grups Descoberts



Clustering vs Segmentació

El clustering és el procés tècnic i automàtic de K-means (o altres algorismes) que simplement agrupa les dades en clústers numèrics (Cluster 0, 1, 2...) sense cap significat de negoci, basant-se només en similituds matemàtiques com distàncies i la inèrcia.

En canvi, la segmentació és el procés complet de negoci que aplica clustering + interpretació humana + estratègia: analitza què representa cada clúster i defineix accions concretes de màrqueting (amb unes conclusions i uns objectius clars).

Així doncs podem associar el *clustering* a tècnica emprada i *segmentació* a la interpretació associada al negoci.

Pasos de la SEGMENTACIÓ: 1. CLUSTERING: Hem trobat 4 grups amb K-Means 2. ANÀLISI: Hem estudiat les característiques de cada grup 3. NOMENCLATURA: Hem posat noms amb significat (Premium, Bàsics...) 4. ESTRATÈGIA: Hem definit què fer amb cada segment (ofertes, sistema de punts de fidelització, facilitats de finançament, etc.).

Exemple Pingüins de l'Antàrtida

Aquestes dades les hem utilitzat en altres exemples i sabem que disposen d'etiquetes (espècie) però les usarem per provar si K-means descobreix les 3 espècies i comprovar els resultats amb la realitat.

```
[ ]: # Carregar pingüins (i netejar dades)
penguins = sns.load_dataset('penguins')
penguins_clean = penguins.dropna()
```

```
[ ]: penguins_clean.head()
```

```
[ ]: species      island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen         39.1           18.7           181.0
1  Adelie  Torgersen         39.5           17.4           186.0
2  Adelie  Torgersen         40.3           18.0           195.0
4  Adelie  Torgersen         36.7           19.3           193.0
5  Adelie  Torgersen         39.3           20.6           190.0

    body_mass_g      sex
0       3750.0    Male
1       3800.0  Female
2       3250.0  Female
4       3450.0  Female
5       3650.0    Male
```

```
[ ]: #Features que utilitzarem
penguins_clean[['species', 'bill_length_mm', 'bill_depth_mm',
                  'flipper_length_mm', 'body_mass_g']].head()
```

```
[ ]: species  bill_length_mm  bill_depth_mm  flipper_length_mm  body_mass_g
0  Adelie         39.1           18.7           181.0           3750.0
1  Adelie         39.5           17.4           186.0           3800.0
2  Adelie         40.3           18.0           195.0           3250.0
4  Adelie         36.7           19.3           193.0           3450.0
5  Adelie         39.3           20.6           190.0           3650.0
```

```
[ ]: # Seleccióem 2 features per poder visualitzar les dades
X_penguins = penguins_clean[['bill_length_mm', 'flipper_length_mm']].values

# Etiquetes per comparar (en un cas real no les tindrem!)
y_true = penguins_clean['species'].astype('category').cat.codes.values
species_names = penguins_clean['species'].astype('category').cat.categories.
↳ tolist()
```

```
[ ]: # Normalitzem les dades
scaler_peng = StandardScaler()
X_penguins_scaled = scaler_peng.fit_transform(X_penguins)
```

A continuació es mostra una animació del procés de formació dels clusters a través d'un valor de k definit (k=3).

- Iteració 0: Centres aleatoris inicials
- Iteracions 1-10: Els centres es mouen i els punts canvien de color
- Final: Convergència (centres estables)

```
[ ]: from matplotlib.animation import FuncAnimation
from IPython.display import HTML

# Configurar la figura
fig, ax = plt.subplots(figsize=(8, 5))

# Centres inicials
initial_centers = np.array([[0, 0], [1, 1], [-1, 1]]) #els marquem nosaltres

# Llista per guardar l'estat de cada iteració
iterations = []
max_iterations = 10

for iter_num in range(max_iterations + 1):
    if iter_num == 0:
        # Iteració 0: només centres inicials
        iterations.append({
            'iter': 0,
            'labels': np.full(len(X_penguins_scaled), -1), # tots grisos
            'centers': initial_centers,
            'title': 'Iteració 0: Centres Aleatoris Inicials'
        })
    else:
        # Aplicar K-Means amb max_iter=iter_num
        kmeans_temp = KMeans(n_clusters=3, init=initial_centers, n_init=1,
                             max_iter=iter_num, random_state=42)
        labels = kmeans_temp.fit_predict(X_penguins_scaled)
        centers = kmeans_temp.cluster_centers_

        if iter_num == 1:
            title = f'Iteració 1: Primera Assignació de Clusters'
        elif iter_num < 5:
            title = f'Iteració {iter_num}: Recalculant Centres'
        elif iter_num < max_iterations:
            title = f'Iteració {iter_num}: Convergint...'
        else:
            title = f'Iteració {iter_num}: CONVERGÈNCIA! '

        iterations.append({
            'iter': iter_num,
            'labels': labels,
            'centers': centers,
            'title': title
        })

# Funció d'animació
def update(frame):
```

```

ax.clear()

state = iterations[frame]
labels = state['labels']
centers = state['centers']

# Pintar punts
if state['iter'] == 0:
    # Iteració 0: tots grisos
    ax.scatter(X_penguins_scaled[:, 0], X_penguins_scaled[:, 1],
               c='gray', s=60, alpha=0.6, edgecolors='black', linewidth=1)
else:
    # Iteracions següents: colors per cluster
    ax.scatter(X_penguins_scaled[:, 0], X_penguins_scaled[:, 1],
               c=labels, s=60, cmap='viridis', alpha=0.6,
               edgecolors='black', linewidth=1)

# Pintar centres
ax.scatter(centers[:, 0], centers[:, 1],
           c='red', s=500, marker='X', edgecolors='black',
           linewidth=3, label='Centres', zorder=5)

# Configurar gràfica
ax.set_xlim(X_penguins_scaled[:, 0].min() - 0.5, X_penguins_scaled[:, 0].
↪max() + 0.5)
ax.set_ylim(X_penguins_scaled[:, 1].min() - 0.5, X_penguins_scaled[:, 1].
↪max() + 0.5)
ax.set_xlabel('Bill Length (normalitzat)', fontsize=12)
ax.set_ylabel('Flipper Length (normalitzat)', fontsize=12)
ax.set_title(state['title'], fontsize=14, fontweight='bold', pad=15)
ax.legend(loc='upper right', fontsize=11)
ax.grid(alpha=0.3)

# Afegir text amb número d'iteració
ax.text(0.02, 0.98, f"Iteració: {state['iter']}",
        transform=ax.transAxes, fontsize=16, fontweight='bold',
        verticalalignment='top',
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8))

# Crear animació
anim = FuncAnimation(fig, update, frames=len(iterations),
                     interval=800, repeat=True, blit=False)

# Mostrar animació
plt.close()
HTML(anim.to_jshtml())

```

```
#Si vols guardar-la com a GIF (opcional):
#anim.save('kmeans_animation.gif', writer='pillow', fps=1)
```

```
[ ]: <IPython.core.display.HTML object>
```

En l'animació podem observar: * Iteració 0: Punts grisos, centres aleatoris (X vermelles) * Iteració 1: Punts prenen colors segons el centre més proper * Iteracions 2-9: Els centres (X) es mouen cap al centre de cada color * Iteració 10: CONVERGÈNCIA - els centres ja no es mouen! Normalment convergeix a les 5-10 iteracions.

Observa com els punts canvien de color quan els centres es mouen durant el procés.

```
[ ]: # Comparem la realitzat amb els resultats optinguts

kmeans_peng = KMeans(n_clusters=3, random_state=42, n_init=10)
y_pred = kmeans_peng.fit_predict(X_penguins_scaled)
centers_peng = scaler_peng.inverse_transform(kmeans_peng.cluster_centers_)

# Visualització comparativa
fig, axes = plt.subplots(2, 1, figsize=(16, 16))

# Espècies reals
for i, species in enumerate(species_names):
    mask = y_true == i
    axes[0].scatter(X_penguins[mask, 0], X_penguins[mask, 1],
                    s=100, alpha=0.7, edgecolors='black', linewidth=1.5,
                    label=species)
axes[0].set_title('ESPÈCIES REALS (Etiquetes conegudes)',
                  fontsize=13, fontweight='bold')
axes[0].set_xlabel('Longitud del Bec (mm)', fontsize=11)
axes[0].set_ylabel('Longitud de les Aletes (mm)', fontsize=11)
axes[0].legend(fontsize=10)
axes[0].grid(alpha=0.3)

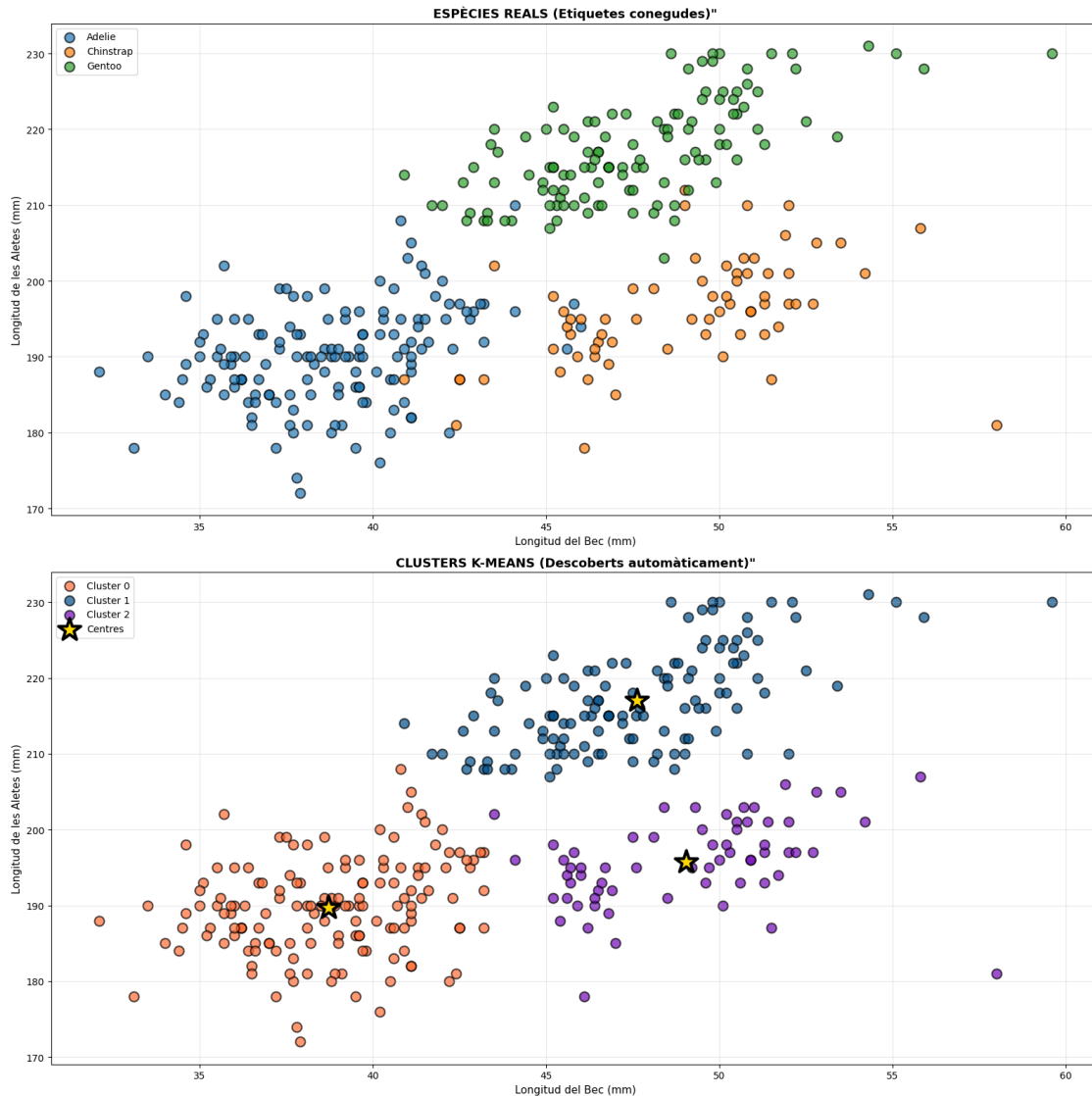
# Clusters K-Means
colors_peng = ['#FF6B35', '#004E89', '#7209B7']
for i in range(3):
    mask = y_pred == i
    axes[1].scatter(X_penguins[mask, 0], X_penguins[mask, 1],
                    s=100, alpha=0.7, edgecolors='black', linewidth=1.5,
                    c=colors_peng[i], label=f'Cluster {i}')
axes[1].scatter(centers_peng[:, 0], centers_peng[:, 1], c='gold', s=600,
                marker='*', edgecolors='black', linewidth=3, label='Centres',
                zorder=5)
axes[1].set_title('CLUSTERS K-MEANS (Descoberts automàticament)',
                  fontsize=13, fontweight='bold')
axes[1].set_xlabel('Longitud del Bec (mm)', fontsize=11)
```

```

axes[1].set_ylabel('Longitud de les Aletes (mm)', fontsize=11)
axes[1].legend(fontsize=10)
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()

```



K-Means ha trobat 3 grups que coincideixen força bé amb les espècies reals, però recorda que en un cas real d'unsupervised learning NO tindríem les etiquetes (espècies). Aquesta comparació és només per “validar” que l'algoritme funciona correctament com exemple acadèmic.

Com podem avaluar un model de clustering?

En supervised learning teníem accuracy, precision, recall i altres mètriques però en *unsupervised*

learning no tenim etiquetes per comparar i validar els resultats. Com sabem si el model clustering és bo?

Tenim un seguit de mètriques que ens poden ajudar, que ja hem vist:

- INERTIA (Inèrcia): suma de distàncies de cada punt al seu centre. • Valor més petit, millor és l'algoritme resultant (grups més compactes) i el seu valor sempre baixa amb més K (cal utilitzar el Elbow Method).
- SILHOUETTE SCORE (0 a 1) mesura com de ben separat està cada punt. • Com més proper a 1 millor (ok a partir de >0.5)
- ELBOW METHOD : representació gràfica d'inertia vs K. • Consisteix en buscar el “colze” (on deixa de millorar significativament amb l'augment de k). Mètode visual i una mica subjectiu.